



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
MATEMATICA

Università degli Studi di Padova
Dipartimento di Matematica "Tullio Levi-Civita"
Corso di Laurea Magistrale in Informatica

Progetto di Intelligenza Artificiale:
Constraint Satisfaction Problems

Relazione - Luglio 2021
Intelligenza Artificiale
Eleonora Signor, 1237581

Indice

1	Informazioni generali	1
2	Constraint Satisfaction Problems	2
2.1	Introduzione al problema	2
2.2	Soddisfacente o insoddisfacente	2
2.3	Scopo della ricerca	2
2.4	Esempi di applicazione	2
2.5	Sistemi e linguaggi di programmazione	3
3	Choco-Solver	4
3.1	Il solver	4
3.2	Studio della libreria	4
3.3	Implementazione di <i>n - Queens problem</i>	4
3.4	Implementazione di <i>Map Coloring problem</i>	8
3.5	Costruzione di un vincolo	10
3.5.1	MyPropagator	11
3.5.2	MyPropagatorComplex	11
4	Le mie considerazioni	13
5	Riferimenti	14

1 Informazioni generali

La presente è la mia relazione, con la quale descrivo i punti e i risultati più importanti inerenti al progetto, che ho realizzato per il corso, di Intelligenza Artificiale.

La suddetta l'ho realizzata parallelamente all'implementazione del codice presente all'interno della *directory progettoCSP*, e si compone di 4 sezioni:

1. **Constraint Satisfaction Problems:** dove descrivo in modo sintetico che cosa è il Problema di Soddisfacimento di Vincoli e quali sono i principali *solver* attualmente in commercio;
2. **Choco-Solver:** che contiene la descrizione operativa del progetto, dove ho riportato e commentato i risultati che ho ottenuto;
3. **Le mie considerazioni:** dove riassumo cosa ho imparato da questa esperienza;
4. **Riferimenti:** dove elenco le risorse di cui ho preso visione durante il mio lavoro.

I modelli gli ho implementati ed eseguiti in *Visual Studio Code*.

2 Constraint Satisfaction Problems

2.1 Introduzione al problema

Un *Constraint Satisfaction Problem* (Problema di Soddisfacimento di Vincoli o CSP) consiste in una ricerca, in cui lo stato si compone di un particolare assegnamento di valore alle variabili.

In tale classe di problemi lo stato non viene visto come una *black box*, comune invece nei problemi di ricerca *standard*¹, difatti un CSP:

- si compone di stati definiti da variabili X_i con valori del dominio D_i ;
- prevede un *test* di *goal* che definisce un insieme di vincoli;
- si compone di un insieme di vincoli che limitano i valori che le variabili possono assumere simultaneamente;
- è un esempio di linguaggio di rappresentazione formale;
- permette di definire algoritmi *general purpose* più potenti degli algoritmi di ricerca *standard*.

2.2 Soddisfacente o insoddisfacente

Una soluzione di un CSP si dice **soddisfacente** quando vi è l'assegnazione di un valore, dal suo dominio a ogni variabile, in modo tale che ogni vincolo sia soddisfatto. Al contrario, se non c'è un'assegnazione di valori alle variabili, dai rispettivi domini, tale che tutti i vincoli siano soddisfatti, il problema è **insoddisfacente**.

2.3 Scopo della ricerca

In generale quando si cerca una soluzione a un Problema con Vincoli, si può essere interessati a individuare:

- una soluzione, senza preferenza su quale;
- tutte le soluzioni;
- una soluzione che sia ottima, o almeno buona, data una funzione oggettiva, definita in termini di alcune o tutte le variabili.

2.4 Esempi di applicazione

Alcuni problemi CSPs, appartenenti al mondo reale, sono legati all'assegnamento per esempio di *tasks*, di orario o di chi insegna a lezione; oppure sono legati alla definizione della data in cui si deve svolgere una particolare attività; o ancora alla configurazione *hardware*; o a problemi di logistica e di schedulazione di attività industriali.

In generale, ogni volta in cui si ha un problema dove si deve pianificare qualcosa, un CSP può essere la soluzione.

¹La ricerca *standard* prevede una qualunque struttura dati in grado di supportare il *test* di *goal*, la funzione di valutazione e la funzione successore.

2.5 Sistemi e linguaggi di programmazione

A oggi sono molti i sistemi e i linguaggi di programmazione esistenti in grado di sfruttare la tecnologia dei vincoli.

Alcuni dei *solver* più usati sono:

- **Choco:** libreria *Java* per Problemi di Soddisfacimento di Vincoli (CSPs), Programmazione di Vincoli (CPs) e Risoluzione dei Vincoli basata sulla Spiegazione (e-CPs);
- **Cream:** libreria di classi che aiuta i programmatori *Java* a sviluppare programmi intelligenti che richiedono la soddisfazione dei vincoli o l'ottimizzazione su domini finiti;
- **Gecode:** ambiente accessibile ed efficiente per lo sviluppo di sistemi e applicazioni basati su vincoli in *C++*;
- **FaCiLe:** libreria di programmazione con vincoli su domini interi finiti scritta in *OCaml*;
- **Chips (*Constraint Handling in Prolog*):** ambiente completo per la progettazione e lo sviluppo di sistemi di supporto decisionale;
- **Python Constraints:** risolutore di problemi di soddisfazione dei vincoli scritto in *Python*.

Inoltre in molte di queste librerie e/o documentazioni (esempio *Gecode*, *Choco-Solver*, ecc..), vengono già inclusi dei modelli di programmazione per la soluzione dei problemi più comuni come: il problema delle 8 - Regine, del Commesso Viaggiatore (TSP), problemi legati all'ubicazione di magazzino e molti altri.

Il mio progetto è incentrato nello studio di *Choco-Solver* con implementazione di qualcuno di questi problemi.

3 Choco-Solver

3.1 Il solver

Choco-Solver è un *software open-source* scritto in *Java*, con lo scopo di descrivere problemi combinatori reali e di risolverli con tecniche di programmazione con vincoli.

Tale *solver* si presta bene sia per l'insegnamento, in quanto facile da usare, che per la ricerca, perchè facile da estendere.

Nel mio progetto ho usato "choco-solver-4.10.6" con dipendenze su libreria *Java* 8.

3.2 Studio della libreria

La parte iniziale del mio lavoro ha riguardato lo studio della documentazione di libreria. Mi sono concentrata soprattutto nella comprensione di:

1. **tipi di variabili:** BoolVar, IntVar, SetVar, RealVar e come usarli;
2. **tipi e dichiarazione di vincoli:** su variabili intere, reali ed espressioni;
3. **metodi di ricerca:** predefinita, personalizzata e locale.

Questo studio preliminare mi è stato vitale per comprendere come costruire modelli in grado di fornire soluzioni corrette.

3.3 Implementazione di *n - Queens problem*

Un problema classico dell'informatica, che può venire risolto come CSP, è il problema delle *n - Regine*. Per questo ho ritenuto interessante costruirmi un modello e verificare la bontà della soluzione individuata dall'algoritmo di ricerca.

Il modello *model_nQueens*, che ho implementato e che si trova posizionato all'interno del *package codice.chocoCode.NQueens*, prevede l'inserimento, da parte dell'utente (come si vede in Figura 1.):

1. del numero di regine;
2. e della strategia di ricerca, che si vuole impiegare, per estrarre variabili e assegnare valori.

```
Inserisci numero di regine (numero intero positivo): 8
Inserisci la strategia di ricerca che vuoi utilizzare: 0
Dati inseriti: 8 0
```

Figura 1: *n - Queens problem*: inserimento numero di regine e strategia di ricerca.

La libreria offre 13 euristiche di ordinamento delle variabili. Ho utilizzato:

- **defaultSearch²**: la quale essendo che non ha alcuna strategia di ricerca indicata esplicitamente, crea una ricerca di *default* per il modello.
- **domOverWDegRefSearch**: la quale realizza una strategia di assegnazione che seleziona una variabile secondo *DomOverWDeg* e la assegna al suo limite inferiore. *DomOverWDeg* è un' euristica di ordinamento delle variabili, per *IntVar*, che seleziona la variabile non istanziata con il rapporto più piccolo $\frac{|d(x)|}{w(x)}$, dove il nominatore denota la dimensione del dominio di una variabile x e il denominatore il suo grado ponderato. Il grado ponderato di un vincolo, si basa su quante volte la propagazione fallisce durante un istante t . Questo viene inizializzato a 1 e aumentato al successore ogni volta che ciò accade.
- **intVarSearch**: la quale applica un'euristica di ricerca di variabili intere. Ho scelto di usare la strategia **FirstFail**, che sceglie la variabile con il dominio più piccolo.
- **activityBasedSearch**: la quale realizza una strategie di ricerca basata sulle attività. Con la quale a ciascuna variabile viene associato un contatore che misura l'attività della stessa durante la propagazione.

Inoltre sono presenti in *Choco* anche 6 euristiche per la selezione del valore. Io ho testato cosa accade nel caso in cui la strategia scelta è **IntDomainMiddle** con *ceiling*: selezione del valore, nel dominio della variabile, più vicino alla media dei suoi limiti attuali, arrotondando per eccesso.

Ho analizzato cosa accade, nel mio modello, con ognuna della strategie di ricerca che ho definito sopra:

1. quando n è pari a 8;
2. chiamando il metodo *printStatistics()*;

La modalità di lettura della soluzione del modello è la seguente: se $Q_j = i$ significa che la regina j va posizionata nella colonna $j + 1$ riga i per non avere conflitti con le altre $n - 1$ regine, considerando di avere una scacchiera $n \times n$ inizializzata a 1.

- **con strategia di ricerca defaultSearch (input da tastiera 0)**: tempo di *building* 0.074s ³, tempo di risoluzione 0.052s ⁴, vengono creati 6 nodi ⁵, numero di fallimenti, *backtracks* e riavvii 0 (Figura 2).

²Nominativo non indicato esplicitamente in *Choco*, ma che impiego per rendere più semplice il riferimento a questa tecnica.

³Da `new Model()` a `solve()`.

⁴Da `new Model()`.

⁵Numero di nodi nella ricerca ad albero binario: 1 per il nodo radice e tra 1 e 2 per ogni decisione (2 quando la decisione è stata rifiutata).

```

Strategia di ricerca predefinita defaultSearch
** Choco 4.10.6 (2020-12) : Constraint Programming Solver, Copyright (c) 2010-2020
- Model[8-queens problem] features:
  Variables : 16
  Constraints : 84
  Building time : 0,074s
  User-defined search strategy : no
  Complementary search strategy : no
- Complete search - 1 solution found.
  Model[8-queens problem]
  Solutions: 1
  Building time : 0,074s
  Resolution time : 0,052s
  Nodes: 6 (115,3 n/s)
  Backtracks: 0
  Backjumps: 0
  Fails: 0
  Restarts: 0
Solution: Q 0=7, Q 1=4, Q 2=2, Q 3=5, Q 4=8, Q 5=1, Q 6=3, Q 7=6,

```

Figura 2: n - Queens problem: soluzione con con strategia di ricerca *defaultSearch*.

- con strategia di ricerca *domOverWDegRefSearch* (input da tastiera 1): tempo di *building* 0.073s, tempo di risoluzione 0.020s, vengono creati 6 nodi, numero di fallimenti, *backtracks* e riavvii 0 (Figura 3).

```

Strategia di ricerca domOverWDegRefSearch
** Choco 4.10.6 (2020-12) : Constraint Programming Solver, Copyright (c) 2010-2020
- Model[8-queens problem] features:
  Variables : 16
  Constraints : 84
  Building time : 0,073s
  User-defined search strategy : yes
  Complementary search strategy : no
- Complete search - 1 solution found.
  Model[8-queens problem]
  Solutions: 1
  Building time : 0,073s
  Resolution time : 0,020s
  Nodes: 6 (294,7 n/s)
  Backtracks: 0
  Backjumps: 0
  Fails: 0
  Restarts: 0
Solution: Q 0=7, Q 1=4, Q 2=2, Q 3=5, Q 4=8, Q 5=1, Q 6=3, Q 7=6,

```

Figura 3: n - Queens problem: soluzione con con strategia di ricerca *domOverWDegRefSearch*.

- con strategia di ricerca *intVarSearch* con *FirstFail* e *IntDomain-Middle* (input da tastiera 2): tempo di *building* 0.069s, tempo di risoluzione 0.013s, vengono creati 7 nodi, numero di fallimenti 3, *backtracks* 4 e riavvii 0 (Figura 4).

```

Strategia di ricerca intVarSearch con FirstFail e IntDomainMiddle
** Choco 4.10.6 (2020-12) : Constraint Programming Solver, Copyright (c) 2010-2020
- Model[8-queens problem] features:
  Variables : 16
  Constraints : 84
  Building time : 0,069s
  User-defined search strategy : yes
  Complementary search strategy : no
- Complete search - 1 solution found.
  Model[8-queens problem]
  Solutions: 1
  Building time : 0,069s
  Resolution time : 0,013s
  Nodes: 7 (543,6 n/s)
  Backtracks: 4
  Backjumps: 0
  Fails: 3
  Restarts: 0
Solution: Q 0=5, Q 1=3, Q 2=8, Q 3=4, Q 4=7, Q 5=1, Q 6=6, Q 7=2,

```

Figura 4: n - *Queens problem*: soluzione con con strategia di ricerca *intVarSearch* con *FirstFail* e *IntDomainMiddle*.

- con strategia di ricerca *activityBasedSearch* (input da tastiera **3**): tempo di *building* 0.069s, tempo di risoluzione 0.031s, vengono creati 65 nodi, numero di fallimenti 13, *backtracks* 72 e riavvii a 14 (Figura 5).

```

Strategia di ricerca activityBasedSearch
** Choco 4.10.6 (2020-12) : Constraint Programming Solver, Copyright (c) 2010-2020
- Model[8-queens problem] features:
  Variables : 16
  Constraints : 84
  Building time : 0,069s
  User-defined search strategy : yes
  Complementary search strategy : no
- Complete search - 1 solution found.
  Model[8-queens problem]
  Solutions: 1
  Building time : 0,069s
  Resolution time : 0,031s
  Nodes: 65 (2.092,9 n/s)
  Backtracks: 72
  Backjumps: 0
  Fails: 13
  Restarts: 14
Solution: Q 0=5, Q 1=1, Q 2=8, Q 3=6, Q 4=3, Q 5=7, Q 6=2, Q 7=4,

```

Figura 5: n - *Queens problem*: soluzione con con strategia di ricerca *activityBasedSearch*.

In conclusione, per il problema 8 - *Queens* la strategia che sembra comportarsi meglio è:

- se considero esclusivamente la minimizzazione dei tempi di *building* e di risoluzione: *intVarSearch* con *FirstFail* e *IntDomainMiddle*.
- se considero la minimizzazione del numero di nodi creati, fallimenti, *backtracks* e riavvii, privilegiando un minor tempo di risoluzione rispetto al *building*: *domOverWDegRefSearch*.

In ogni caso ognuna delle ricerche è in grado di individuare una configurazione di regine che non ha conflitti nella scacchiera.

3.4 Implementazione di *Map Coloring problem*

Un altro problema classico dell'informatica, che può venire risolto con CSP, consiste nel *Map Coloring*. Ovvero la colorazione della mappa dell'Australia sulla base di tre colori (verde, rosso, blu), in cui ogni Stato ha un colore diverso da ogni altro Stato a lui confinante.

Le modalità di lettura delle soluzioni dei modelli sono le seguenti: WA = Western Australia, NT = Northern Territory, SA = South Australia, NSW = New South Wales, Q = Queensland, V = Victoria e T = Tasmania (rappresentano gli Stati dell'Australia); 1 = rosso; 2 = blu, 3 = verde (i colori di assegnamento).

Con riferimento al *package codice.chocoCode.MapColoring*, ho realizzato i seguenti modelli:

- *model_mapColoring_defaultsearch*: con cui ho trovato tutte le soluzioni possibili al problema del *Map Coloring* con strategia di ricerca **defaultSearch**⁶ (Figura 6).

```
Strategia di ricerca predefinita defaultSearch

** Choco 4.10.6 (2020-12) : Constraint Programming Solver, Copyright (c) 2010-2020
- Model[Map Coloring] features:
  Variables : 8
  Constraints : 18
  Building time : 0,088s
  User-defined search strategy : no
  Complementary search strategy : no
- Complete search - 18 solution(s) found.
  Model[Map Coloring]
  Solutions: 18
  Building time : 0,088s
  Resolution time : 0,058s
  Nodes: 35 (605,4 n/s)
  Backtracks: 35
  Backjumps: 0
  Fails: 0
  Restarts: 0
Solution: WA=1, NT=2, SA=3, Q=1, NSW=2, V=1, T=1,
Solution: WA=1, NT=2, SA=3, Q=1, NSW=2, V=1, T=2,
Solution: WA=1, NT=2, SA=3, Q=1, NSW=2, V=1, T=3,
Solution: WA=1, NT=3, SA=2, Q=1, NSW=3, V=1, T=1,
Solution: WA=1, NT=3, SA=2, Q=1, NSW=3, V=1, T=2,
Solution: WA=1, NT=3, SA=2, Q=1, NSW=3, V=1, T=3,
Solution: WA=2, NT=1, SA=3, Q=2, NSW=1, V=2, T=1,
Solution: WA=2, NT=1, SA=3, Q=2, NSW=1, V=2, T=2,
Solution: WA=2, NT=1, SA=3, Q=2, NSW=1, V=2, T=3,
Solution: WA=2, NT=3, SA=1, Q=2, NSW=3, V=2, T=1,
Solution: WA=2, NT=3, SA=1, Q=2, NSW=3, V=2, T=2,
Solution: WA=2, NT=3, SA=1, Q=2, NSW=3, V=2, T=3,
Solution: WA=3, NT=2, SA=1, Q=3, NSW=2, V=3, T=1,
Solution: WA=3, NT=2, SA=1, Q=3, NSW=2, V=3, T=2,
Solution: WA=3, NT=2, SA=1, Q=3, NSW=2, V=3, T=3,
Solution: WA=3, NT=1, SA=2, Q=3, NSW=1, V=3, T=1,
Solution: WA=3, NT=1, SA=2, Q=3, NSW=1, V=3, T=2,
Solution: WA=3, NT=1, SA=2, Q=3, NSW=1, V=3, T=3,
```

Figura 6: *Map Coloring problem*: tutte le soluzioni con strategia di ricerca *defaultSearch*.

⁶Per la definizione di *defaultSearch* vedi *n - Queens problem*.

Inoltre da alcuni *test*, sempre sul medesimo modello, ho ottenuto che la soluzione ottima viene trovata in 0.015s, con 0 riavvii, 7 *backtracks* e 3 fallimenti (Figura 7).

```
- Complete search - 1 solution found.
  Model[Map Coloring]
  Solutions: 1
  MINIMIZE WA = 1,
  Building time : 0,071s
  Resolution time : 0,042s
  Time to best solution : 0,015s
  Nodes: 4 (95,1 n/s)
  Backtracks: 7
  Backjumps: 0
  Fails: 3
  Restarts: 0
  Solution: WA=1, NT=2, SA=3, Q=1, NSW=2, V=1, T=1,
```

Figura 7: *Map Coloring problem*: *test* con strategia di ricerca *defaultSearch*.

- *model_mapColoring_localsearch*: in questo caso ho trovato la migliore soluzione ottima, usando un algoritmo di **ricerca locale di grandi quartieri**.

Vista la dimensione limitata del problema (7 variabili per 3 valori di dominio), e che già l'uso della ricerca di *default* permette di trovare la soluzione ottima in poco tempo, ho ritenuto sufficiente usare il metodo di ricerca **setLNS**, con lo scopo di ridurre il numero di *backtracks*. Questo adopera un *INeighborFactory* che fornisce configurazioni predefinite, senza alcuna assunzione particolare sulla combinazione dei vicini; inoltre ho vincolato la ricerca al riavvio dopo 100 fallimenti.

Una volta terminata la ricerca, ho estratto la migliore soluzione ottima. Per fare questo ho dovuto definire la variabile sulla quale ottimizzare. Le variabili che danno i risultati in tempo di risoluzione e *building* migliori sono : *WA*, *T*, *SA* e *NT*. In ogni caso gli scostamenti sono minimi.

Se a essere ottimizzata è *WA* la migliore soluzione viene trovata in 0.015s circa, con 1 riavvio, 4 *backtracks* e nessun fallimento (Figura 8). Da notarsi come la soluzione trovata coincide con quella di *defaultSearch*.

```

Strategia di ricerca LNS casuale

** Choco 4.10.6 (2020-12) : Constraint Programming Solver, Copyright (c) 2010-2020
- Model[Map Coloring] features:
  Variables : 8
  Constraints : 18
  Building time : 0,098s
  User-defined search strategy : no
  Complementary search strategy : no
- Complete search - 1 solution found.
  Model[Map Coloring]
  Solutions: 1
  MINIMIZE WA = 1,
  Building time : 0,098s
  Resolution time : 0,040s
  Time to best solution : 0,015s
  Nodes: 4 (99,5 n/s)
  Backtracks: 4
  Backjumps: 0
  Fails: 0
  Restarts: 1
  Solution: WA=1, NT=2, SA=3, Q=1, NSW=2, V=1, T=1,

```

Figura 8: *Map Coloring problem*: soluzione ottima con strategia di ricerca LNS

In conclusione la ricerca locale confrontata con una ricerca generica non porta grandi cambiamenti, nè nel tempo di individuazione nè nella soluzione ottima trovata.

Però è chiaro, dai risultati che ho ottenuto, come nella ricerca locale, il fatto che si vada di volta in volta a migliorare una soluzione parziale, porti come conseguenza una riduzione del numero di *backtracks* con aumento del numero di riavvii. Quest'ultimo non è considerarsi come un fatto esclusivamente negativo, in quanto può, di caso in caso, incentivare l'individuazione della soluzione ottima. Inoltre anche il numero di fallimenti, ovvero conflitti, rispetto a *defaultSearch*, vengono azzerati.

3.5 Costruzione di un vincolo

Oltre a realizzare modelli per la soluzione di problemi, mi sono anche occupata di costruirmi un vincolo "personale", nello specifico della forma $\sum_{i=0}^{n-1} x_i \leq limit$ (vincolo di somma).

Il numero di incognite della disequazione (espressione a sinistra) n e il limite superiore della stessa (valore di destra) $limit$ sono entrambi trattati come *input* forniti dall'utente, mediante terminale.

Ho prodotto i seguenti *file* posizionati all'interno del *package codice.chocoCode.ConstraintDesign*:

- *main*: in cui ho implementato un modello che fa uso del vincolo di somma;
- *MyPropagator*: in cui ho implementato la classe *MyPropagator*;
- *MyPropagatorComplex*: in cui ho implementato la classe *MyPropagatorComplex*.

Uno o più propagatori definiscono un vincolo. Il caso studio che ho realizzato usa un propagatore *MyPropagatorComplex*.

3.5.1 MyPropagator

La mia prima implementazione del propagatore. Si compone di:

- **un costruttore:** i parametri sono *limit*, che indica il limite di valore richiesto dalla soluzione del vincolo, e *x* array di variabili;
- **public void propagate(int evtmask):** che si occupa di applicare l'algoritmo di filtraggio⁷. Questo viene fatto mediante:
 1. il calcolo della variabile locale *F*, differenza tra il *limit* e la somma di tutti i *lower-bounds* delle variabili;
 2. la riduzione dell'*upper-bound* della variabile, quando oltre il *bound* richiesto da *limit*.
- **public int getPropagationConditions(int vIdx):** ritorna il tipo di evento che si scatena a seguito di una modifica di variabile. In questo caso si tratta o di istanziazione o di limite inferiore crescente.
- **public ESat isEntailed():** stima se il propagatore è già stato coinvolto nella ricerca degli assegnamenti alle variabili.

3.5.2 MyPropagatorComplex

Qui ho svolto una progettazione di vincoli più complessa rispetto a *MyPropagator*. Le modifiche rispetto alla classe base riguardano:

- **l'aggiunta dei parametri:** *F*, *backtrackable int* che permette di registrare sia *F* che il *lower-bound* delle variabili e *prev_lbs*, array che memorizza il precedente *lower-bound* delle variabili;
- **public int getPropagationConditions(int vIdx):** in cui viene trattato anche l'evento di senza alcuna modifica (utile per il metodo *propagate*);
- **i metodi *propagate* sono due:**
 - **public void propagate(int evtmask):** chiama il metodo di supporto *void prepare()*, il quale istanzia i valori di *F* e *prev_lbs*, e aggiorna gli *upper-bounds* delle variabili, se necessario.
 - **public void propagate(int vIdx, int evtmask):** si occupa di aggiornare *F* con la differenza tra l'attuale e il precedente *lower-bound*, al fine di calcolare *F* in modo incrementale, e svolgere così una propagazione degli eventi parziale.

Ho definito che l'individuazione di una soluzione, a un problema, avvenga mediante ricerca predefinita (*defaultSearch*).

Di seguito ho riportato due tra i risultati che ho ottenuto, durante i miei *test*. Figura 9 mostra un caso in cui la scelta di *n* e *limit* rende impossibile trovare una soluzione; Figura 10 invece presenta un caso contrario.

⁷Un algoritmo di filtraggio ha come obiettivo eliminare dal suo dominio di variabili, i valori che non possono appartenere ad alcuna soluzione.

```
Inserisci numero di variabili del vincolo (numero intero positivo): 8
Inserisci limite del vincolo: -1
Il numero di variabili del vincolo e': 8
Il limite del vincolo e': -1
Non esiste alcuna combinazione di domini di variabili tale che il vincolo '<= -1' possa venire rispettato
```

Figura 9: *Vincolo di somma*: soluzione insoddisfacente.

```
Inserisci numero di variabili del vincolo (numero intero positivo): 8
Inserisci limite del vincolo: 15
Il numero di variabili del vincolo e': 8
Il limite del vincolo e': 15
Solution: x0=1, x1=1, x2=1, x3=1, x4=1, x5=1, x6=1, x7=1,
```

Figura 10: *Vincolo di somma*: soluzione soddisfacente.

4 Le mie considerazioni

Grazie a questo progetto ho avuto la possibilità di:

- innanzitutto riprendere in mano *Java*, linguaggio che non utilizzavo da due anni, e di impiegarlo anche con l'aggiunta di quelle nozioni che in questo intervallo di tempo ho acquisito;
- costruire dei modelli che mi hanno permesso di comprendere, non solo sulla teoria, come effettivamente deve funzionare un algoritmo di soddisfacimento di vincoli;
- comprendere i limiti del cervello umano. Ogni soluzione che ho trovato l'ho verificata personalmente; e anche solo su un problema relativamente piccolo come *Map Coloring*, più di una volta mi sono "persa", durante tale compito, e la soluzione mi era in ogni caso fornita da un algoritmo.
- implementare un propagatore e con questo comprendere come viene definito nel concreto un vincolo, all'interno di una libreria che tratta la classe di problemi in esame. Questo mi ha permesso anche di "uscire" dal formalismo matematico, dato dall'esclusiva definizione del problema.
- di comprendere come un vincolo semplice come la somma, abbia bisogno di una dichiarazione di classe e almeno 4 metodi. E di come dunque una libreria semplifichi la vita dei programmatori.

Sicuramente l'area di ricerca dei CSPs è un ambito molto interessante, che contiene ancora molte sfide. Mi sarebbe piaciuto approfondire ulteriormente l'argomento, per esempio provando a trasformare un problema in un grafo, ma per limiti di tempo non mi è stato possibile.

In ogni caso, anche se il mio lavoro è stato limitato e ristretto nel formulare modelli dell'informatica classica, mi sono comunque accorta di come i CSPs possano essere integrati nella vuota quotidiana di ognuno di noi. Per esempio possono aiutarci a capire:

- come sistemare i calzini nel cassetto in modo da non usare sempre gli stessi;
- come distribuire i nostri impegni durante la settimana, in modo da minimizzare i tempi morti, e magari non arrivare in ritardo agli appuntamenti;
- come ridipingere la nostra casa, se si hanno più piani e si vuole che le stanze seguano una nostra personale strategia di colore;
- come riempire i letti di un ospedale, in modo da velocizzare le ammissioni e le dimissioni dei pazienti;
- e molti altri ...

5 Riferimenti

Durante lo sviluppo del progetto ho preso visione delle seguenti fonti:

- <https://ktiml.mff.cuni.cz/~bartak/constraints/systems.html>;
- <https://choco-solver.org/>;
- https://elearning.unipd.it/math/pluginfile.php/89626/mod_resource/content/3/CSP.pdf, *slides* pag. 298-300.